SUTD Computer Vision (50.035) Final Report

Group 16

Lee Le Xuan, Keith Low, Issac Koh, Aqif Yeo

December, 2024

Abstract

Understanding block diagrams is an often-overlooked challenge that requires complex functions such as deep-learning based computer vision and object detection methods. While these methods are effective for symbol detection, their reliance on bounding box representations poses challenges in recognizing line segments that are either very short or nearly parallel to the axes. In this report, we propose an improvement to the model developed by Bhushan and Lee by presenting a change in the Inception-ResNet-v2 backbone used in their paper. We compared the performances of using a Swin transformer and also introduced a new dataset for a wider variety of values.

1. Introduction

Block diagrams are a key medium to display complicated relationships or processes in an easy-to-read manner. These block diagrams have become more and more prevalent, often being included in user manuals, scientific papers and business process documentation. With a wider adoption of block diagrams in the current literature to display these complex situations, further flowchart recognition development is required for document analysis and recognition [4].

Block diagrams comprise of many complex shapes, lines or arrows that add to the complexity of understanding them using Document Artificial Intelligence (Document AI). They range from simple systems with few processes to highly interconnected subsystems, joined by numerous components. This poses a challenge for Document AI, which has many applications for document classification [7], information extraction [5] and visual question answering (source). As a result, those with visual impairments may struggle to understand block diagrams, due to the difficult nature of recognizing and refining the structural semantics of block diagrams [9].

In this paper, we propose a Swin transformer backbone to improve performance, with the goal of improving the work of Shreyanshu and Lee [2]. The core idea of the improvement is to replace the Faster RCNN backbone proposed in their architecture, more specifically within the shape prediction of block diagrams. A block diagram generator was implemented along side the possible improvements, to add further complexity and variety for the model training.

2. Related Work

2.1. Block Diagram Recognition

Most prior studies of machine-generated block diagram recognition took place after the CLEF-IP in 2011, which was held by the Information Retrieval Facility (IFR) and released a public machine-generated flowchart dataset [13]. Adams [1] extracted the entire block diagram by analysing the connected components in images, before identifying specific structures using manually chosen features. This however would result in reduced accuracy when detecting incomplete (dotted lines) or broken edges. Mörzinger et al. [10] first separated the diagram and text before studying the visual features of a structure, going to the pixel level.

For handwritten block diagrams, Julca-Aguilar and Hirata[6] employed the Faster R-CNN for symbol detection. Results show that Faster R-CNN was effective on both publicly available flowchart and mathematical expression (CROHME-2016) datasets. Schäfer [11] built the first deep learning system for joint structure and symbol recognition in handwritten diagrams using Arrow R-CNN.

On a similar vein, Farahani and colleagues [3] reviewed recent automatic chart image understanding techniques more focused on charts (bar chart, pie chart, scatter plot). Shreyanshu and Lee [2] proposed a new framework for the summarization of block diagram images, called “Blosum”. The “Blosum” architecture would decompose the images into possible sets of triplets, capturing the relationships between components.

2.2. Transformer Architecture

Transformer architectures have revolutionized computer vision tasks in recent years, with models like

the Vision Transformer (ViT) and its derivatives achieving state-of-the-art performance across various domains. Unlike traditional convolutional neural networks (CNNs), which process spatially local information using kernels, transformers utilize self-attention mechanisms to capture global context within an image. This makes them particularly suited for recognizing structural relationships in block diagrams, where global context plays a vital role in understanding component interactions and spatial dependencies.

The Swin Transformer [8], a hierarchical vision transformer, was introduced as a means to overcome the computational challenges associated with traditional transformer models. By adopting a shifted window mechanism, Swin Transformer processes images in smaller, non-overlapping windows while allowing information flow between adjacent windows. This approach ensures a fine balance between computational efficiency and the ability to model long-range dependencies. Such a capability is critical for block diagram recognition, where short and nearly parallel line segments often require an understanding of their global positioning within the diagram.

The rationale behind exploring the Swin Transformer for block diagram recognition lies in its ability to handle hierarchical representations effectively. Additionally, its adaptability to irregular shapes and non-uniform layouts aligns well with the inherent variability in block diagram structures.

3. Proposed Approach

3.1. Motivation

Of all the papers that we examined, we were interested in the work done by Bhushan and Lee [2] on block diagram summarization capabilities. Although Blosum performed respectably well for the online generated block diagrams, it struggles mainly with hand-written texts. This was due to the version of EasyOCR not supporting handwritten texts and the focus of Blosum being on computerized block diagrams. Hence, our project would focus on improving the Blosum architecture to better detect handwritten text, to generate a better performance on the FC_A dataset.

Based on this, our project centers around modifying the existing architecture presented by Bhushan and Lee [2] to better detect handwritten block diagrams. Blosum is comprised of mainly four parts, as shown in the Figure 1. It consists of shape prediction, text prediction, arrow prediction and triplet generation. We propose that by experimenting with the backbone of the Blosum architecture, we may be able to derive better results

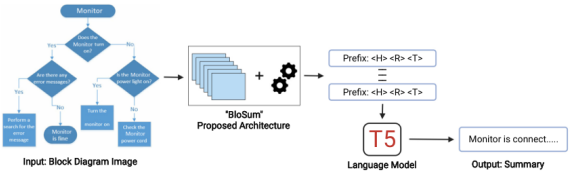

Figure 1: Blosum architecture proposed by Bhushan and Lee [2].

with regards to the shape prediction, leading to a better performance on the FC_A dataset.

3.2. Our Model

Our architecture model explored in this project followed the architecture used Bhushan and Lee [2]. Focusing on the shape prediction portion, we substituted the Inception-ResNet-v2 backbone for a Swin transformer instead.

The choice of the Swin transformer was due to its ability to deal with small visual entities and pixel level patterns. The Swin transformer can address these challenges by proposing hierarchical processing of image tokens using shifted window patches with linear complexity to image size [12]. This hierarchical architecture has the flexibility to model at various scales and has linear computational complexity with respect to image size [8], allowing greater performance when detecting arrowheads in block diagrams.

3.3. Shape Prediction and Triplet Generation

The shape prediction feature was treated as an object detection task. Similar to Bhushan and Lee [2], for each shape $s \in S$, it predicts a bounding box $bs \in R4$ and a class name $cs \in C$. We defined seven plus one (background) different classes, which include Connection, Data, Decision, Terminator, and Process.

The Inception-ResNet-v2 backbone here worked by having multi-level feature extraction that can capture information at various scales and complexities. Dimensionality reduction and pooling was also carried out to reduce the complexity and reduce over fitting respectively. To ensure fair testing, we also kept the similar intersection over union (IoU) threshold value of 0.8 over all shape classes. To prevent duplicates, a non-maximum suppression (NMS) was applied.

Additionally, based on the information collected from the previous steps, we created a similar pipeline used to form a triplet generator. The generator would capture the relationship between the shapes detected by finding the closest anchor point placed on other shapes.

4. Dataset

4.1. Dataset Used

To ensure fairness in the improvement of our model, we opted to use the same computerized block diagram that was presented by the authors. This new dataset comprised of 502 diagrams and contained more than 13,000 annotated elements (shapes, edges, and text phrases) and was retrieved on GitHub. There are total 7 classes, each for a block element: Connection for circle, Data for parallelogram, Decision for diamond, Terminator for eclipse, Arrow, Text and Process for all other shapes not mentioned above.

4.2. Data Augmentation

As highlighted previously, a key limiting factor that mitigated the performance of the original Inception-ResNet-v2 model was the difficulty of obtaining useful annotated data sets for object detection models. Despite scraping 502 images online, the manual task of annotating them with detailed labels (shapes, arrows, texts), bounding-boxes, and creating human-written summaries is labour-intensive and time-consuming.

Additionally, the 502 images had limited variability as shown in Figure 2. The dataset would not capture the full diversity of block diagrams, including different shapes, shape representations, text styles, and arrow connections. In this work, we propose to a data generation plan with an existing knowledge base and Mermaid graphing tool. With an additional new dataset, it would be able to help improve the generalization capabilities of our model compared to other datasets.

This ideally allows models to learn how to reverse engineer the entity-relation-entity triplets from the generated diagrams. For each entity-relation-entity triplet $x = [e_1, r, e_2]$, construct a directed graph $G = (V, E)$ such that there exists vertices $e_1, e_2 \in V$, a directed edge $(e_1, e_2) \in E$ with label r for all $x \in X$, where X represents set of all triplets in the knowledge graph. An pseudo-code version of this algorithm is shown in Figure 3. Finally, visualize this graph using certain visualization tools such as Mermaid, and easily obtain the labels and bounding-boxes for the diagram because of it's computerized nature.

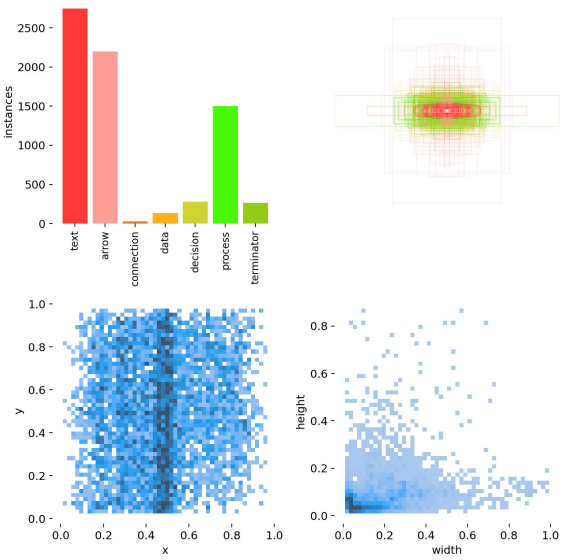

Figure 2: Class Instances of the CBD Dataset

```
function GRAPH(root)
    queue ← [root]
    explored ← {root}
    graph ← {}
    exploration ← RANDOMNUMBER(4, 10)
    while queue not empty and explored.length < exploration do
        parent ← DEQUEUE(queue)
        children ← GETRELATIONCHILDPAIRS(parent)
        sampledChildren ← SAMPLECHILDREN(children)
        for each relation, child in sampledChildren do
            if child not in explored do
                ENQUEUE(queue, child)
                explored ← explored + {child}
            graph ← graph + {(parent, relation, child)}
    return graph
```

Figure 3: Pseudo-code for the graph generation algorithm

5. Implementation

5.1. Testing

We trained on the test set of the computerized block diagrams, which had 300 images, and evaluated on FC-A, which is the handwritten dataset. This is to test out the generalizability of our models, and is also inline with testing done by the paper Blosum.

Table 1: Mean Average Precision per class score matrix on FC_A

	Faster RCNN - Swin T	Faster RCNN - Inception-ResNet-v2
Text	0.44	0.25
Arrow	0.41	0.36
Connection	0.11	0.00
Data	0.75	0.46
Decison	0.70	0.54
Process	0.63	0.61
Terminator	0.85	0.07

Table 2: Mean Average Recall per class score matrix on FC_A

	Faster RCNN - Swin T	Faster RCNN - Inception-ResNet-v2
Text	0.80	0.69
Arrow	0.81	0.74
Connection	0.31	0.05
Data	0.85	0.67
Decison	0.84	0.72
Process	0.75	0.79
Terminator	0.88	0.54

5.2. Evaluation Metrics

For our evaluation, we calculated the F1-Score, Mean Average Precision (MAP) and Mean Average Recall (MAR). We set IOU=0.7, similar to the paper.

These are the results we obtained are summarised in table 1, table 2 and table 3.

5.3. Results

The overall MAP of the model with Swin-Transformer backbone was higher than that of with the Inception-ResNet-v2 backbone, even though the Inception-ResNet-v2 managed to achieve a slightly higher MAP on the test set of the computerized block diagrams. This shows that the Swin-Transformer is able to generalize better. In addition, the overall training duration of the Swin-Transformer was twice as fast as that of the Inception-ResNet-v2.

For both Inception and Swin-T models, the precision and recall for "Connection" are the lowest. This could be due to having less data for the "Connection" shapes, and we might not have ran enough epochs on both models to capture enough information.

The MAP for the models are generally quite low as we only ran 50 epochs on both. This was due to time and resource constraints.

Overall, the model with Swin-T as the backbone performed better.

For the data augmentation, we ran it 50 sets of it with 20 epochs on the Swin Transformer. Results was better by 0.02 MAP, and we can extrapolate this to 50 epochs to predict that the results would be better.

6. Limitations and Challenges

The detection of small or nearly parallel line segments, especially in crowded or dense block diagrams, remains a challenge. Despite the Faster RCNN's ability to capture fine-grained details, certain small features, such as tiny arrows or short lines, are still difficult to detect accurately. This issue is exacerbated in diagrams where these features are highly variable or not aligned to standard orientations.

Although the augmented dataset increased the variety of diagrams, there are still many non-standard diagram formats and visual representations that our model struggles to recognize effectively. Diagrams with unconventional shapes, inconsistent use of arrows, or text positioned outside the typical areas may lead to misclassification or detection failures.

7. Sustainable Development Goals

Our work presented in this project pushes Sustainable Development Goals number four and sixteen, Quality Education and Peace, Justice and Strong Institutions. With an easier way to summarize block diagrams through visual document understanding, this allows for automated content digitisations for greater accessibility to all.

We believe that this would allow for better educational experiences for visually impaired learners through text descriptions or audio narration. Additionally, improving on process documentation can lead to the advancement of stronger institutes.

Table 3: Confusion Matrix on *FC_A*

	Faster RCNN - Swin T	Faster RCNN - Inception-ResNet-v2
Precision (MAP)	0.57	0.34
Recall (MAR)	0.77	0.62
F1 Score	0.66	0.44

Having an easier way to understand flowcharts can lead to easier management and efficient decision making.

8. Work Distribution

Le Xuan - Implementation of the Inception-ResNet-v2 backbone for benchmark testing, implementation of the Swin transformer model, performed evaluation metrics for both backbones.

Keith Low - Data augmentation for the generation of a new dataset, implementation of the YOLOv5 backbone for benchmark testing, performed evaluation metrics for YOLOv5.

Isaac Koh - Research into Triplet generation, implementation of triplet with the Inception-ResNet-v2 backbone

Aqif Yeo - Report formatting and structure, research on implementation of Inception-ResNet-v2 backbone for benchmark testing.

9. Conclusion

This report aimed to improve the performance of an existing model by Bhushan and Lee to detect shapes in block diagrams. Our proposed model replaced the Inception-ResNet-v2 backbone model with a Swin transformer and a YOLOv5 model in the hopes of scoring a better F-measure, Precision and Recall score. After testing with our own generated dataset, only the YOLOv5 model (98.4) yielded better results (96.24). This could be attributed to the one step interence in YOLOv5 compared to the two-step in Faster R-CNN. Although performing better, recognizing block diagrams and understanding the corresponding structural semantics with an end to-end model is still a hurdle.

References

[1] S. Adams, "Electronic non-text material in patent applications—some questions for patent offices, applicants and searchers", World Patent Information **27**, 99–103 (2005) 10.1016/j.wpi.2004.12.005.

[2] S. Bhushan and M. Lee, "Block diagram-to-text: understanding block diagram images by generating natural language descriptors", in Findings of the association for computational linguistics: aacl-ijcnlp 2022, edited by Y. He, H. Ji, S. Li, Y. Liu, and C.-H. Chang (Nov. 2022), pp. 153–168, 10.18653/v1/2022.findings-aacl.15.

[3] A. V. Farahani, P. Adibi, A. Darvishy, M. S. Ehsani, and H.-P. Hutter, "Automatic chart understanding: a review", IEEE Access **11**, 76202–76221 (2023) 10.1109/access.2023.3298050.

[4] D. J. Griffiths, *Introduction to electrodynamics*, Fourth edition (Cambridge University Press, 2017), ISBN: 9781108420419.

[5] W. Hwang, S. Kim, M. Seo, J. Yim, S. Park, S. Park, J. Lee, B. Lee, and H. Lee, "Post-ocr parsing: building simple and robust parser via bio tagging", in Workshop on document intelligence at neurips 2019 (2019).

[6] F. D. Julca-Aguilar and N. S. T. Hirata, "Symbol detection in online handwritten graphics using faster R-CNN", CoRR **abs/1712.04833** (2017).

[7] L. Kang, J. Kumar, P. Ye, L. Yi, and D. Doermann, "Convolutional neural networks for document image classification", 10.1109/icpr.2014.546 (2014) 10.1109/icpr.2014.546.

[8] Z. Liu, Y. Lin, Y. Cao, J. Qiu, Y. Wei, Z. G. Zhang, S. Lin, and B. Guo, "Swin transformer: hierarchical vision transformer using shifted windows", arXiv (Cornell University), 10.48550/arxiv.2103.14030 (2021) 10.48550/arxiv.2103.14030.

[9] M. Mathew, D. Karatzas, and C. Jawahar, "Docvqa: a dataset for vqa on document images", arXiv (Cornell University), 10.48550/arxiv.2007.00398 (2020) 10.48550/arxiv.2007.00398.

[10] R. Mörzinger, R. Schuster, A. Horti, and G. Thallinger, *Visual structure analysis of flow charts in patent images*, 2012.

[11] B. Schäfer, M. Keuper, and H. Stuckenschmidt, "Arrow r-cnn for handwritten diagram recognition", **24**, 3–17 (2021) 10.1007/s10032-020-00361-1.

[12] A. Singh, *Swin transformers explained — layer architecture more*, Bolster AI, Jan. 2024.

[13]L. Sun, H. Du, and T. Hou, "Fr-detr: end-to-end flowchart recognition with precision and robustness", IEEE Access **10**, 64292–64301 (2022) 10.1109/ACCESS.2022.3183068.

Appendix A. Triplet Generation

We attempted to generate Head, Relation, Tail ($< H >< R >< T >$) triplets following the model proposed in BloSum [2].

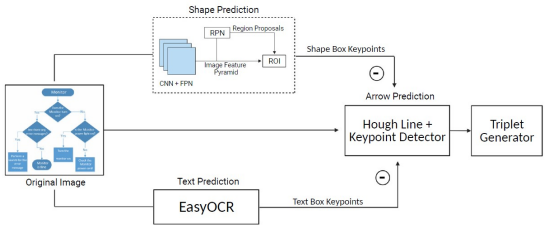

Figure A.4: Overall Architecture proposed in BloSum

Appendix A.1. Text Prediction

We used EasyOCR (Jaided, 2020) for detecting text T in an image. For each text $t \in T$, it predicts a bounding box $b_t \in \mathbb{R}^4$. We combined texts t where t's bounding box falls within the bounding box of a shape s.

```
Shape 9:
  Text: Analog Image
  Average Confidence: 0.9926
  BB Coord: [194.06809997558594, 63.28110122680664, 312.176513671875, 150.44540405273438]
  Label: 6
Shape 10:
  Text: Driver
  Average Confidence: 0.9659
  BB Coord: [364.62469482421875, 63.90449905395508, 485.7013854980469, 149.9947052001953]
  Label: 6
Shape 11:
  Text: Camera
  Average Confidence: 0.9488
  BB Coord: [49.585201263427734, 71.27980041503906, 141.93299865722656, 138.9250030517578]
  Label: 6
Shape 12:
  Text: Digital Image
  Average Confidence: 0.9704
  BB Coord: [533.1923828125, 61.01210021972656, 669.0, 156.35650634765625]
  Label: 6
Shape 13:
  Text: Output
  Average Confidence: 0.9998
  BB Coord: [536.4385986328125, 217.25880432128906, 669.0, 306.2059020996094]
  Label: 6
```

Figure A.5: Sample Output for EasyOCR

Appendix A.2. Arrow Prediction

We subtracted non-arrow classes from the image before binarizing it to leave only the arrows behind. We then applied the Hough Line Transform to detect break and connecting arrows. We predicted the head and tail of an arrow by counting white pixels at the start and end points.

Appendix A.3. Triplet Generator Logic

We attempted to apply the same triplet generator logic used in BloSum [2].

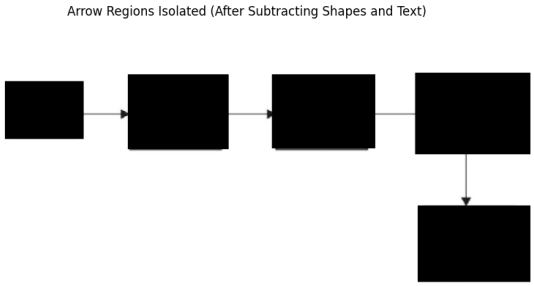

Figure A.6: Image after subtracting non-arrow classes

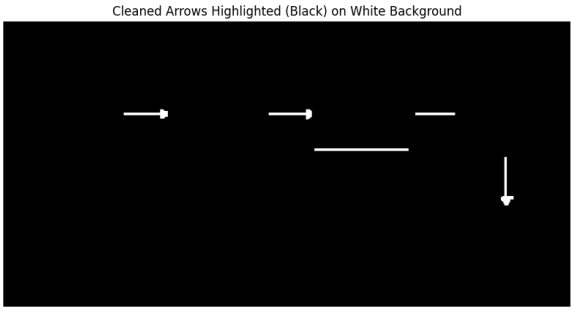

Figure A.7: Image after Binarization

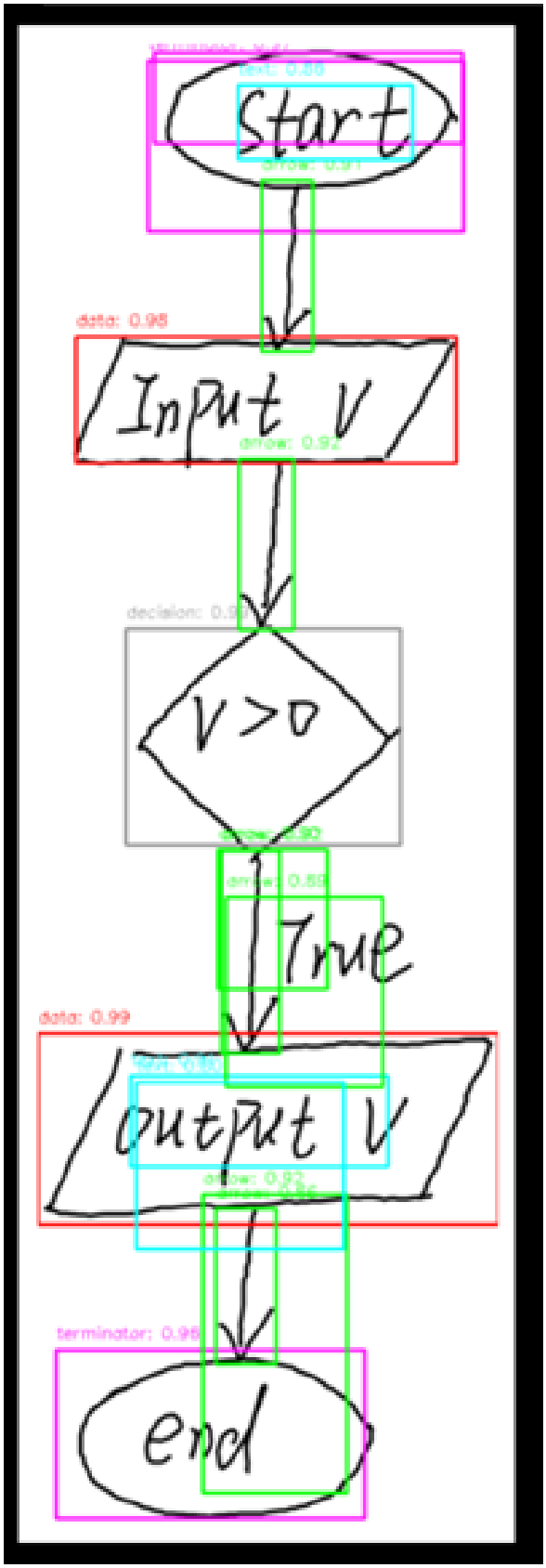

Figure A.8: Fig Swin T CBD 50 epoch

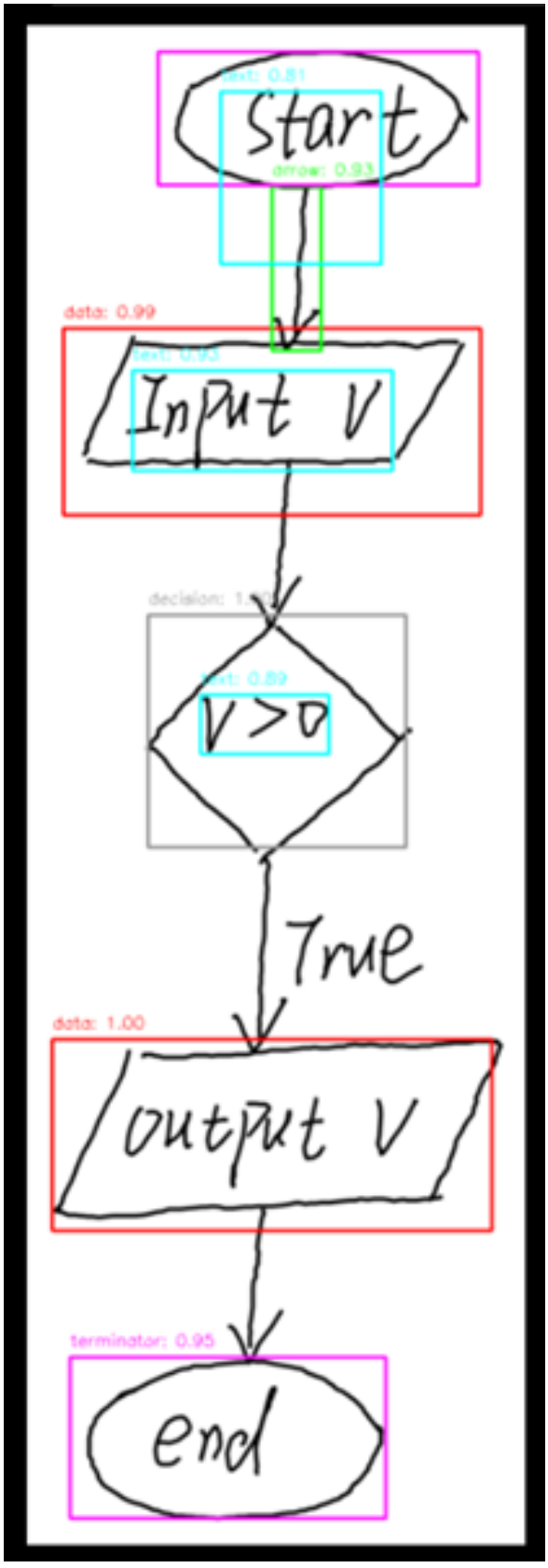

Figure A.9: Swin t 100 epoch fca

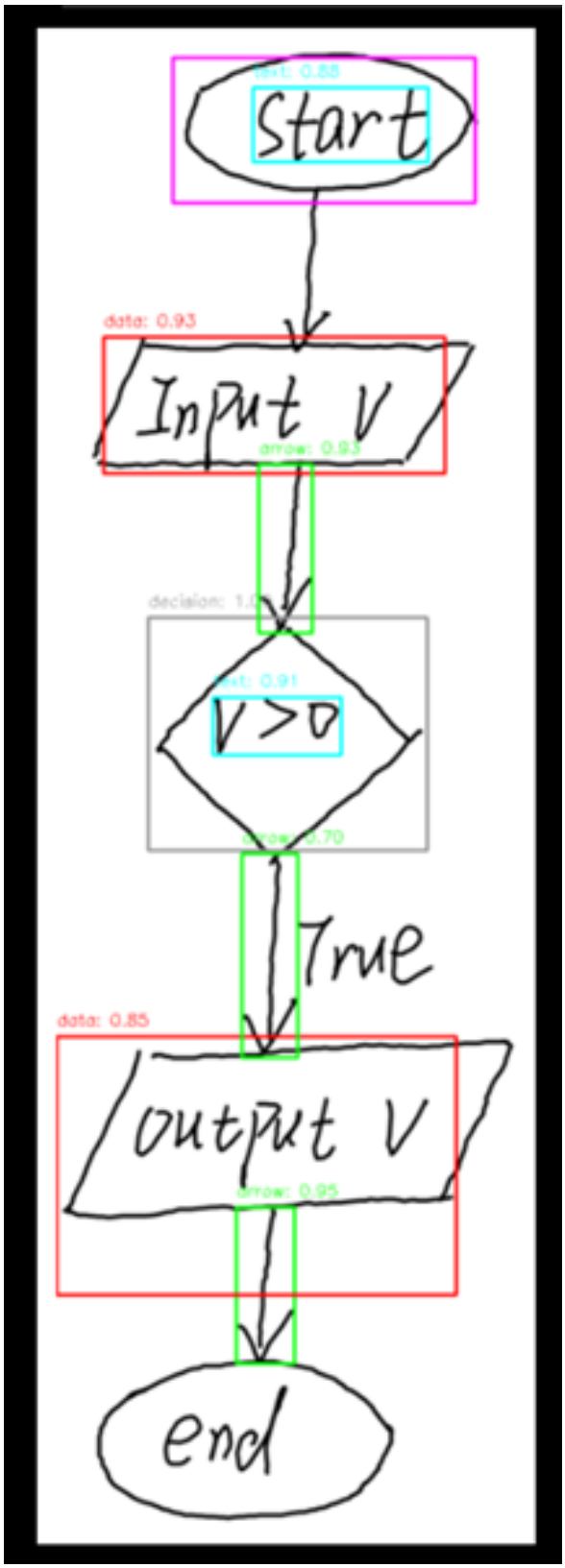

Figure A.10: Inception Resnet 50 epochs